

Project 29

Multi-class SVM, a Lagrangian dual of the representer identity,
and a bundle method

Tolgonai Nasipbek kyzy (626449)

Hamse Hassan Adnan (683187)

Abstract

(M) is a kernel Support Vector Machine for multi-class classification built with the standard One-vs-Rest construction. (A1) is a Lagrangian dual approach applied to the *dual* formulation of the SVM: we reintroduce the weight vector w as an explicit variable, so that the representer identity $w = \sum_i \alpha_i y_i \phi(x_i)$ becomes a constraint, and we dualise *that* identity. The resulting Lagrangian relaxation is a linear program over a box plus one equality constraint — a continuous knapsack, solved exactly by sorting in $O(m \log m)$ — so the dual function is finite everywhere, genuinely non-differentiable, and defined on a space of dimension n (linear kernel) or m (general kernel). It is maximised by a proximal bundle method, whose master problem is handed to an off-the-shelf QP solver. (A2) is a general-purpose convex QP solver applied to the standard SVM dual.

1 The standard SVM

1.1 Primal

Let $\{(x_i, y_i)\}_{i=1}^m$ with $x_i \in \mathbb{R}^n$ and $y_i \in \{-1, +1\}$, and let $\phi : \mathbb{R}^n \rightarrow \mathcal{H}$ be a feature map into a Hilbert space with reproducing kernel $\kappa(x, x') = \langle \phi(x), \phi(x') \rangle$. Write $\phi_i := \phi(x_i)$. The soft-margin (C-)SVM is

$$\begin{aligned} \min_{w \in \mathcal{H}, b \in \mathbb{R}, \xi \in \mathbb{R}^m} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i \\ \text{s.t.} \quad & y_i (\langle w, \phi_i \rangle + b) \geq 1 - \xi_i, \quad i = 1, \dots, m, \\ & \xi_i \geq 0, \quad i = 1, \dots, m. \end{aligned} \tag{P}$$

The first term maximises the geometric margin $1/\|w\|$ between the two supporting hyperplanes $\langle w, \phi(x) \rangle + b = \pm 1$; the second penalises, linearly and with weight $C > 0$, the amount by which each sample fails to reach its own margin. Problem (P) is a convex QP; it is strictly convex in w but only convex in (b, ξ) .

Eliminating ξ gives the equivalent unconstrained, non-smooth form

$$\min_{w, b} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m [1 - y_i (\langle w, \phi_i \rangle + b)]^+, \quad [t]^+ := \max\{0, t\}, \tag{1}$$

which we shall meet again, unexpectedly, in Section 3.5.

1.2 Wolfe dual

Attach $\alpha_i \geq 0$ to the margin constraints and $\mu_i \geq 0$ to $\xi_i \geq 0$:

$$L(w, b, \xi; \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \mathbf{1}^\top \xi + \sum_{i=1}^m \alpha_i (1 - \xi_i - y_i (\langle w, \phi_i \rangle + b)) - \mu^\top \xi.$$

Stationarity in (w, b, ξ) gives the three conditions

$$w = \sum_{i=1}^m \alpha_i y_i \phi_i, \quad \sum_{i=1}^m \alpha_i y_i = 0, \quad \alpha_i + \mu_i = C. \quad (2)$$

The first is the *representer identity*; it is the hinge on which this whole project turns — Section 3 dualises precisely this condition.

Substituting the three conditions back into L makes every term collapse. Writing $K_{ij} = \kappa(x_i, x_j)$, $Y = \text{diag}(y)$ and $Q = YKY$:

$$\begin{aligned} \frac{1}{2} \|w\|^2 &= \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij} = \frac{1}{2} \alpha^\top Q \alpha, \\ - \sum_i \alpha_i y_i \langle w, \phi_i \rangle &= - \left\langle w, \sum_i \alpha_i y_i \phi_i \right\rangle = - \|w\|^2 = - \alpha^\top Q \alpha, \\ -b \sum_i \alpha_i y_i &= 0 \quad (\text{by the } b \text{ condition}), \\ \sum_i \xi_i (C - \alpha_i - \mu_i) &= 0 \quad (\text{by the } \xi \text{ condition}), \end{aligned}$$

leaving $\mathbf{1}^\top \alpha - \frac{1}{2} \alpha^\top Q \alpha$, to be maximised. Two remarks on how the constraints of the dual arise, because both are easy to state loosely:

- $\mu_i \geq 0$ and $\mu_i = C - \alpha_i$ together give $\alpha_i \leq C$, which with $\alpha_i \geq 0$ is the box $0 \leq \alpha \leq C\mathbf{1}$;
- b enters L only through the term $-b \sum_i \alpha_i y_i$, and b is *free*, so $\inf_b L = -\infty$ unless $\sum_i \alpha_i y_i = 0$. The b -stationarity condition therefore does not disappear when it is used to cancel that term: it survives as the constraint $y^\top \alpha = 0$, which is exactly the statement that dom of the dual function excludes every α violating it.

Writing the result as a minimisation of the negative, the dual of (P) is the convex QP

$$\min_{\alpha \in \mathbb{R}^m} \quad \frac{1}{2} \alpha^\top Q \alpha - \mathbf{1}^\top \alpha \quad \text{s.t.} \quad \alpha \in D := \{\alpha : y^\top \alpha = 0, \quad 0 \leq \alpha \leq C\mathbf{1}\}. \quad (\text{D})$$

D is a non-empty ($0 \in D$) compact polytope and $Q \succeq 0$, so (D) always has a solution. Slater's condition holds for (P), hence there is no duality gap and the KKT conditions

$$\alpha_i = 0 \Rightarrow y_i f(x_i) \geq 1, \quad 0 < \alpha_i < C \Rightarrow y_i f(x_i) = 1, \quad \alpha_i = C \Rightarrow y_i f(x_i) \leq 1 \quad (3)$$

characterise optimality, where $f(x) = \sum_i \alpha_i y_i \kappa(x_i, x) + b$. Samples with $\alpha_i > 0$ are the *support vectors*; those with $0 < \alpha_i < C$ are the *free* support vectors, and b is recovered from any of them via $b = y_i - \sum_j \alpha_j y_j \kappa(x_j, x_i)$. The count of free support vectors will turn out to control the cost of (A1); see Section 4.4.

1.3 Recovering the primal variables

Given an optimal α^* of (D), the primal variables come back as

$$w^* = \sum_i \alpha_i^* y_i \phi_i, \quad \xi_i^* = [1 - y_i (\langle w^*, \phi_i \rangle + b^*)]^+,$$

the first directly from the representer identity and the second from primal feasibility of the margin constraint together with complementarity on $\xi_i \geq 0$ (note the factor y_i inside: ξ_i measures the violation of *the sample's own* margin, not of the hyperplane equation).

The bias b^* deserves more care, and it is where this report departs from the usual treatment. The textbook recipe reads b^* off any free support vector via (3), and averages over them for stability; but the average is only as good as the set of indices identified as free, so a solver that is merely close to optimal produces a b that degrades the measured primal–dual gap. The better observation — made, for instance, in the interior-point treatment of the same model in Project 25 — is that b enters the Lagrangian only through $-b \sum_i \alpha_i y_i$, so b is the multiplier of the equality constraint $y^\top \alpha = 0$. Any algorithm that produces that multiplier explicitly can use it directly, with no support-vector bookkeeping at all.

For an interior-point method this multiplier is one of the iterates. What is worth noting here is that our relaxation supplies it too, and for free: the multiplier θ of the single equality inside the continuous knapsack of Section 3.3 is b , as (6) proves by LP duality. So (A1) returns the bias at every oracle call rather than only at optimality, and the identity holds by construction rather than by an argument about which α_i are free. We verify this numerically against a general-purpose QP solver to 10^{-7} .

1.4 Kernels

Only inner products appear in (D), so ϕ need never be formed. We use the linear kernel $\kappa(x, x') = x^\top x'$, the Gaussian kernel $\kappa(x, x') = \exp(-\gamma \|x - x'\|^2)$, and the polynomial kernel $\kappa(x, x') = (\gamma x^\top x' + c_0)^d$.

2 One-vs-Rest

With q classes and labels $c_i \in \{1, \dots, q\}$, One-vs-Rest (OvR) solves q independent binary problems. For class k set

$$y_i^k = \begin{cases} +1, & c_i = k \\ -1, & c_i \neq k \end{cases}$$

and solve (P)/(D) with $y = y^k$, obtaining $f^k(x) = \sum_i \alpha_i^k y_i^k \kappa(x_i, x) + b^k$. A new point is assigned to $\arg \max_k f^k(x)$. Note that:

1. The q problems share *the same* kernel matrix K ; they differ only through the label vector, since $Q^k = Y^k K Y^k$ and D^k depends on k only through the single equality $(y^k)^\top \alpha = 0$. Q^k is never formed.
2. The classifiers are trained independently but compared at prediction time on a common scale that nothing in the training enforces; this produces the familiar OvR pathologies, ties ($f^k(x) > 0$ for two k) and rejections ($f^k(x) < 0$ for all k).
3. Each binary problem is unbalanced, roughly $1 : (q - 1)$.

Point 1 is the basis of the discussion in Section 6, which the project statement explicitly requires.

3 Which constraints to dualise

3.1 Three requirements

For a Lagrangian dual approach to be worth building, the relaxation must satisfy:

- (R1) the relaxed problem must be *substantially easier* than the original — ideally closed-form or combinatorial;
- (R2) the dual must be *multi-dimensional*, otherwise a one-dimensional search solves it and bundle machinery is pointless;

(R3) the dual should be *genuinely non-differentiable*, otherwise a smooth method is preferable.

These are exactly the three tests our earlier proposals failed, and they are worth stating because they make the correct choice inevitable.

Relaxing the single equality $y^\top \alpha = 0$ in (D) violates (R1) and (R2): what is left is

$$\min \left\{ \frac{1}{2} \alpha^\top Q \alpha - \mathbf{1}^\top \alpha + \theta y^\top \alpha : 0 \leq \alpha \leq C \mathbf{1} \right\},$$

a box-constrained QP with the *same dense Hessian* — the unbiased SVM, no easier than the original — and the dual is univariate. Relaxing the box $\alpha \leq C \mathbf{1}$ fares no better: the relaxation is again a QP in Q .

The reason is structural, and it is the point we had missed: in (D) *no constraint is the difficulty*. The difficulty is Q itself, a dense $m \times m$ matrix. So the object to dualise is not a constraint of (D) at all — it is the identity that *created* Q .

3.2 Dualising the representer identity

Q appears in (D) only because w was eliminated through the representer identity (2). Put w back:

$$\min_{\alpha, w} \frac{1}{2} \|w\|^2 - \mathbf{1}^\top \alpha \quad \text{s.t.} \quad w = \sum_{i=1}^m \alpha_i y_i \phi_i, \quad \alpha \in D. \quad (\text{D}')$$

Substituting the constraint recovers (D) exactly, so (D') and (D) are the same problem. Now dualise *only* the linking identity, with a free multiplier $\lambda \in \mathcal{H}$:

$$L(\alpha, w; \lambda) = \frac{1}{2} \|w\|^2 - \mathbf{1}^\top \alpha + \left\langle \lambda, w - \sum_i \alpha_i y_i \phi_i \right\rangle.$$

The two groups of variables separate. Minimising over $w \in \mathcal{H}$ is unconstrained and gives $w(\lambda) = -\lambda$ with value $-\frac{1}{2} \|\lambda\|^2$; what remains is linear in α over D . Hence

$$\boxed{\varphi(\lambda) = -\frac{1}{2} \|\lambda\|^2 + \min_{\alpha \in D} c(\lambda)^\top \alpha, \quad c_i(\lambda) = -(1 + y_i \langle \lambda, \phi_i \rangle),} \quad (4)$$

and the Lagrangian dual of (D') is $\max_{\lambda \in \mathcal{H}} \varphi(\lambda)$.

3.3 The relaxed problem is a continuous knapsack

The inner problem in (4) is

$$\min c^\top \alpha \quad \text{s.t.} \quad y^\top \alpha = 0, \quad 0 \leq \alpha \leq C \mathbf{1}, \quad (\text{KP})$$

a linear program over a box with a *single* equality constraint: the classical *continuous knapsack* problem. Relaxing that one equality with a scalar θ makes the box minimiser explicit,

$$\alpha_i(\theta) = \begin{cases} C, & c_i - \theta y_i < 0, \\ 0, & c_i - \theta y_i > 0, \\ \text{any value in } [0, C], & c_i - \theta y_i = 0, \end{cases}$$

and $h(\theta) := y^\top \alpha(\theta)$ is non-decreasing, so a θ^* with $h(\theta^*) = 0$ exists and is found by sorting the m breakpoints $y_i c_i$ and scanning (or by median search in $O(m)$ expected time). Splitting the tied components to make the equality hold exactly produces an α that minimises the Lagrangian of (KP) *and* is feasible, hence optimal. Forming c costs one kernel product; the rest costs $O(m \log m)$. This settles (R1): a dense QP has been replaced by a sort.

3.4 Properties of φ

Proposition 1 (finiteness). *D is a non-empty compact polytope, so the minimum in (4) is attained and finite for every λ . Hence $\varphi : \mathcal{H} \rightarrow \mathbb{R}$ is finite everywhere, concave and upper semicontinuous.*

This is unusually convenient: no feasibility (“infinite”) cuts are ever needed, and every oracle call returns a valid lower bound on the optimal value.

Proposition 2 (structure and non-differentiability). *$\varphi = \varphi_{\text{sm}} + \varphi_{\text{pl}}$ where $\varphi_{\text{sm}}(\lambda) = -\frac{1}{2}\|\lambda\|^2$ is smooth and known in closed form, and $\varphi_{\text{pl}}(\lambda) = \min_{\alpha \in D} c(\lambda)^\top \alpha$ is a minimum of finitely many affine functions of λ (one per vertex of D), hence concave polyhedral. φ fails to be differentiable exactly where (KP) has multiple optima, i.e. where some reduced cost vanishes, $c_i - \theta^* y_i = 0$.*

Unwinding that condition with $w = -\lambda$ gives $y_i \langle w, \phi_i \rangle + \theta^* y_i = 1$: the kinks of φ sit *exactly* on the support-vector condition $y_i f(x_i) = 1$ of (3). Since any non-degenerate SVM has free support vectors, the maximiser of φ is a point of non-differentiability. This settles (R3): the non-smoothness is not incidental, it is the solution.

Proposition 3 (no duality gap). *(D') is convex, its coupling constraint is affine and D is compact; therefore $\max_\lambda \varphi(\lambda) = v(D') = v(D)$, and the maximiser is $\lambda^* = -w^*$.*

The multiplier of the relaxed identity is thus, up to sign, the primal weight vector itself. A supergradient of φ at λ is the violation of the relaxed constraint,

$$g(\lambda) = w(\lambda) - \sum_i \alpha_i(\lambda) y_i \phi_i = -\lambda - \sum_i \alpha_i(\lambda) y_i \phi_i, \quad (5)$$

available at no extra cost from the oracle. (R2) holds as well: λ ranges over $\mathcal{H}$, of dimension n for the linear kernel and m after the reduction of Section 4.3 in general.

3.5 What this dual actually is

It is worth identifying $\max_\lambda \varphi$ in closed form. Substituting $w = -\lambda$ in (4),

$$\max_\lambda \varphi(\lambda) \iff -\min_w \left\{ \frac{1}{2} \|w\|^2 + \max_{\alpha \in D} \sum_i \alpha_i (1 - y_i \langle w, \phi_i \rangle) \right\},$$

and dualising the single equality inside the inner maximisation with a scalar θ gives, by LP duality,

$$\max_{\alpha \in D} \sum_i \alpha_i (1 - y_i g_i) = \min_\theta C \sum_i [1 - y_i (g_i + \theta)]^+, \quad g_i := \langle w, \phi_i \rangle. \quad (6)$$

So the Lagrangian dual of (D') is the hinge-loss primal (1), and *the multiplier θ of the knapsack equality is the SVM bias b* (up to the sign convention of Section 3.3) — so b falls out of the oracle for free, with no need for the usual “average over the free support vectors” recipe. That the dual of the dual is the primal is of course biconjugacy; what is useful here is the algorithmic consequence, namely that (A1) is a bundle method on a function for which published convergence results exist: the cutting-plane and bundle methods for regularised risk minimisation of Joachims (KDD 2006), Teo et al. (JMLR 2010) and Franc & Sonnenburg (ICML 2008) converge in $O(1/\varepsilon)$ iterations, improving to linear under the strong convexity of $\frac{1}{2}\|w\|^2$.

3.6 Relation to bundle methods for regularised risk minimisation

Section 3.5 showed that $\max_\lambda \varphi$ is the hinge-loss primal, so it is fair to ask whether (A1) is anything more than the known cutting-plane / bundle methods applied directly to that primal

(Joachims KDD 2006; Teo et al. JMLR 2010; Franc & Sonnenburg ICML 2008), and in particular whether dualising the representer identity buys anything at all. We state the relationship precisely rather than leave it implicit, because the answer is sharper than it first appears.

Those methods minimise $\frac{\lambda}{2}\|w\|^2 + \frac{1}{m}\sum_i \ell(f_i, y_i)$ and take as their cut the subgradient of the empirical risk, whose support is $\{i : y_i f_i < 1\}$. Our oracle instead returns the knapsack solution $\alpha^*(\lambda)$. These are not merely similar: measured on WINE with a Gaussian kernel at random multipliers, $\alpha^*(\lambda)$ agrees with the plain hinge pattern [C if $y_i f_i < 1$, 0 otherwise] on only 41%–83% of the samples, but agrees with the *biased* pattern [C if $y_i(f_i + b) < 1$] on 100% of them, where $b = -\theta$ is the multiplier the knapsack itself produces. In other words:

the oracle of (A1) is the hinge-loss oracle with the bias optimised out exactly, at every iteration,

and the sorting of Section 3.3 is precisely that one-dimensional minimisation over b .

This is what the dualisation buys, and it is not cosmetic. The cited methods drop the bias term — as does the implementation of Tortorella & Ferrante (**bundle-svm**, 2017), whose decision function is $\text{sign} \sum_i u_i \kappa(x_i, x)$ with no intercept — and they drop it for a reason: with a bias the empirical risk is no longer a plain separable sum, and a closed-form subgradient is no longer available. What restores it is exactly the equality $y^\top \alpha = 0$, which is in D *only because* b was a free variable of (P). Relaxing the representer identity, rather than writing the primal down directly, is what puts that equality in the relaxed problem and hence what makes the biased oracle closed-form.

Two further things follow from the dual route that the direct primal route does not give: the master multipliers return a feasible $\hat{\alpha} \in D$ and therefore the SVM dual variables themselves (Section 4.2), and $\varphi(\lambda)$ and the restricted problem bracket $v(D)$ from both sides, whereas the primal methods bound only their own objective. We claim no more than this: the family of cuts is the same, the function being optimised is the same, and what the Lagrangian formulation adds is the bias, the primal variables and the two-sided bound.

4 (A1): the bundle method

4.1 Model and master problem

Because $-\frac{1}{2}\|\lambda\|^2$ is known in closed form, we linearise only the polyhedral part — a *generalized* (structured) bundle method. Each oracle call returns some $\alpha^j \in D$, which gives the affine upper model

$$\varphi_{\text{pl}}(\lambda) \leq c(\lambda)^\top \alpha^j = -\mathbf{1}^\top \alpha^j - \langle \lambda, \sum_i \alpha_i^j y_i \phi_i \rangle =: -a_j - \langle \lambda, \Phi^\top Y \alpha^j \rangle.$$

With a stability centre $\bar{\lambda}$ and proximal parameter $t > 0$ the master is

$$\max_{\lambda \in \mathcal{H}} -\frac{1}{2}\|\lambda\|^2 + \min_{j=1,\dots,J} \{-a_j - \langle \lambda, b_j \rangle\} - \frac{1}{2t}\|\lambda - \bar{\lambda}\|^2, \quad b_j := \Phi^\top Y \alpha^j. \quad (7)$$

Exchanging min and max (Sion) and eliminating λ gives its dual, a QP over the unit simplex Δ^J :

$$\min_{\vartheta \in \Delta^J} \frac{1}{2\rho} \left\| \frac{\bar{\lambda}}{t} - \sum_j \vartheta_j b_j \right\|^2 - \sum_j \vartheta_j a_j, \quad \rho := 1 + \frac{1}{t}, \quad (8)$$

after which $\lambda^{\text{new}} = (\bar{\lambda}/t - \sum_j \vartheta_j b_j)/\rho$. Only the $J \times J$ Gram matrix $G_{jl} = \langle b_j, b_l \rangle$ and the vector $p_j = \langle b_j, \bar{\lambda} \rangle$ are needed; both are maintained incrementally, so a master costs $O(J^2)$ plus one kernel product per new cut. We solve it with an off-the-shelf QP solver, which the project statement allows.

4.2 Primal recovery, and the Dantzig–Wolfe identity

Let $\hat{\alpha} := \sum_j \vartheta_j \alpha^j$. Since $\vartheta \in \Delta^J$ and D is convex, $\hat{\alpha} \in D$ — it is a *feasible* solution of (D) at every iteration. Moreover $\sum_j \vartheta_j b_j = \Phi^\top Y \hat{\alpha}$ and $\sum_j \vartheta_j a_j = \mathbf{1}^\top \hat{\alpha}$, so for $t \rightarrow \infty$ (8) becomes

$$\min_{\vartheta \in \Delta^J} \frac{1}{2} \hat{\alpha}^\top Q \hat{\alpha} - \mathbf{1}^\top \hat{\alpha},$$

i.e. *the SVM dual (D) restricted to $\text{conv}\{\alpha^1, \dots, \alpha^J\} \subseteq D$* . The method is therefore exactly Dantzig–Wolfe decomposition of (D) with (KP) as pricing problem, and bundle, cutting planes on (1) and column generation on (D) are three views of one algorithm. Two practical consequences:

- the master returns the primal solution $\hat{\alpha} \rightarrow \alpha^*$, so nothing extra is needed to recover the classifier;
- $\varphi(\lambda) \leq v(D) \leq \frac{1}{2} \hat{\alpha}^\top Q \hat{\alpha} - \mathbf{1}^\top \hat{\alpha}$ at *every* iteration, a **certified gap** — which decomposition algorithms for the SVM such as SMO do not provide.

4.3 Kernelised form

For a general kernel $\mathcal{H}$ may be infinite-dimensional, so write $\lambda = \sum_j \beta_j \phi_j$. This is without loss of generality: a component of λ orthogonal to $\text{span}\{\phi_i\}$ leaves $c(\lambda)$ unchanged and only increases $\|\lambda\|^2$, so it is zero at the optimum. Then $\langle \lambda, \phi_i \rangle = (K\beta)_i$ and $\|\lambda\|^2 = \beta^\top K \beta$, and one oracle call costs one matrix–vector product with K plus a sort. Note that the correct stabilising term is the *RKHS* norm $\|\lambda - \bar{\lambda}\|^2 = (\beta - \bar{\beta})^\top K (\beta - \bar{\beta})$ rather than the Euclidean $\|\beta - \bar{\beta}\|^2$; it is the K -metric that makes (8) come out in the form above.

4.4 Cost, and how many cuts are needed

Per iteration: one oracle call ($O(m^2)$ for the kernel product, $O(m \log m)$ for the sort) plus one master ($O(J^2)$ in the QP, $O(Jm)$ to update G). The interesting question is how large J must grow. The oracle returns essentially *vertices* of D , that is $0/C$ vectors, and $\hat{\alpha}$ is a convex combination of them: a component of α^* sitting at a bound is produced by a single vertex, but every *free* support vector $0 < \alpha_i^* < C$ must be produced by mixing vertices that disagree on i . We therefore expect, and measure, that

the number of cuts the method needs is governed by the number of free support vectors,

and that truncating the bundle below that number makes the gap stall — the classical Dantzig–Wolfe tailing-off. This is confirmed sharply in our prototype: on WINE, the linear kernel yields 12 free support vectors and the method converges to a gap of $2 \cdot 10^{-8}$ in 26 iterations, whereas the Gaussian kernel ($\gamma = 0.2$) yields 91 free support vectors out of $m = 178$, and with a bundle capped at 60 cuts the gap stalls at $2 \cdot 10^{-4}$, while raising the cap to 200 restores convergence in 153 iterations. The practical rule is to size the bundle above the expected number of free support vectors, which large C and simple kernels make small.

5 (A2): a general-purpose solver

For comparison, (D) is handed as a dense convex QP to an off-the-shelf interior-point solver. This is the reference both for solution quality (we verify that (A1) and (A2) return the same α^* , b and test accuracy) and for running time.

6 Does solving one One-vs-Rest problem help the others?

The project statement asks this explicitly. The q problems differ only through y^k , so there are three candidate mechanisms, and they are not equally useful.

1. The kernel matrix (real). K is computed once and shared; $Q^k = Y^k K Y^k$ is never formed, since the oracle only ever needs $K\beta$. Better still, the q oracles can be run in lockstep so that their q matrix–vector products become one BLAS-3 product $K[\beta^1 \dots \beta^q]$. This is a genuine, structural saving of a factor q in the dominant cost.

2. Warm-starting the multiplier (measured, and it does not help). Since $\lambda^* = -w^*$, warm-starting class $k+1$ at class k 's optimum amounts to assuming the two separating hyperplanes are similar. They are not: the positive class changes completely between the two problems. In our experiments this is neutral at best and can be clearly harmful (on WINE with a Gaussian kernel it raised the total oracle-call count by 72%), because the bundle inherits a stability centre far from the new optimum and spends iterations undoing it.

3. Transferring cuts (measured, and it is roughly neutral). Any point of D^l generates a valid cut for class l , and the D^k differ only through the equality. Writing P_k, P_l for the two positive blocks and R for the rest, $\alpha \in D^k$ satisfies $\alpha(P_k) = \alpha(P_l) + \alpha(R)$, so $(y^l)^\top \alpha = -2\alpha(R)$: the imbalance is known in closed form and can be repaired in $O(m)$ by moving mass onto P_l , *preserving the support pattern* of the transferred cut. This costs no oracle call and no kernel product. Measured effect: between -4% and $+2\%$ oracle calls — within noise.

The honest conclusion, which we will state as such in the report, is that for this decomposition the transferable quantity is the *kernel matrix*, not the dual solution: mechanism 1 is worth a factor q , while mechanisms 2 and 3 are not worth their complexity. That is a more useful answer than a plausible-sounding claim about warm starts, and it is the one the measurements support.

A Appendix: the shared-slack-budget model, and why we set it aside

For completeness we record the model proposed in our previous submission, now derived properly, together with a defect we found in it.

The model. Couple the q One-vs-Rest problems by giving each sample a single slack budget B_i to be shared by all q binary classifiers:

$$\begin{aligned} \min_{w, b, \xi} \quad & \sum_{k=1}^q \frac{1}{2} \|w^k\|^2 + C \sum_{i,k} \xi_i^k \\ \text{s.t.} \quad & y_i^k (\langle w^k, \phi_i \rangle + b^k) \geq 1 - \xi_i^k, \quad \xi_i^k \geq 0, \quad \sum_{k=1}^q \xi_i^k \leq B_i. \end{aligned} \tag{M-B}$$

Its dual. Dualising only the mq margin constraints with $\alpha_i^k \geq 0$ separates the variables. Minimising over w^k gives $w^k = \sum_i \alpha_i^k y_i^k \phi_i$ and $-\frac{1}{2}(\alpha^k)^\top Q^k \alpha^k$; minimising over b^k forces $\sum_i \alpha_i^k y_i^k = 0$; and minimising over ξ splits across i into the tiny LP $\min\{\sum_k (C - \alpha_i^k) \xi_i^k : \xi_i^k \geq 0, \sum_k \xi_i^k \leq B_i\}$, solved by inspection: put the whole budget on the most negative coefficient, or use none of it. Hence

$$\varphi(\alpha) = \sum_{i,k} \alpha_i^k - \frac{1}{2} \sum_k (\alpha^k)^\top Q^k \alpha^k - \sum_i B_i \left[\max_k (\alpha_i^k - C) \right]^+, \tag{9}$$

on $\{\alpha \geq 0 : \sum_i \alpha_i^k y_i^k = 0 \ \forall k\}$. The budget replaces the box $\alpha \leq C$ of (D) by a penalty, and the dual is non-differentiable where two or more classes tie for a contested budget, or at the kink $\max_k \alpha_i^k = C$ — which is where the One-vs-Rest coupling actually happens.

The defect. With a *hard* budget, (M-B) can be infeasible, and then its Lagrangian dual is unbounded. This is not a corner case: a sample lying where two classifiers both score near zero needs $\xi_i^k \approx 1$ for each of them, so $B_i < 2$ is typically already infeasible. On a 3-class Gaussian blob instance we measured the feasibility threshold at exactly $B_{\min} = 2.0$, with (9) diverging for every $B < B_{\min}$ and strong duality holding to 10^{-8} for every $B \geq B_{\min}$. A single outlier destroys the training problem.

The repair. Soften the budget: $\sum_k \xi_i^k \leq B_i + z_i$ with $z_i \geq 0$ penalised by Dz_i . The ξ subproblem stays an LP solved by inspection, the model is always feasible, and boundedness of the z term restores a box, $0 \leq \alpha \leq (C + D)\mathbf{1}$, making $\text{dom } \varphi$ compact. We verified strong duality for the repaired model to 10^{-8} across a range of B , D , q and m .

Why we nevertheless prefer the approach of Section 3. Even repaired, (9) retains the quadratic forms Q^k *inside* the dual function, so the bundle’s piecewise-linear model has to approximate a curved function and the method degenerates into solving a QP by cutting planes. The relaxation of Section 3, by contrast, leaves an LP and keeps its only curved term in closed form in the master. And (M-B) changes the model (M), so any comparison against One-vs-Rest would confound a modelling change with an algorithmic one, whereas Section 3 leaves (M) exactly the standard One-vs-Rest SVM and changes only how it is solved.